FlyWithLua Dear ImGui Contrib UX & Extensions Guide

FlyWithLua 2.8.18+ / ImgWindow v2.2.1+

Comprehensive Reference for Markdown, TimedTooltip, and Modern Interactive Widgets

---

1. Overview & Architecture

FlyWithLua integrates the modern `ImgWindow v2 contrib/` extension ecosystem directly into the Lua scripting environment. This brings rich typography, intelligent stationary tooltips, smooth animated toggles, reset-enabled sliders, and mouse-wheel chasers to X-Plane floating windows.

All extension bindings are implemented natively in C++ (`src/FloatingWindows/FWLContribBindings.cpp`), exposing zero-overhead wrappers that bridge Lua scripts to `ImgWindow`'s header-only extension library.

Key Capabilities

- **Semantic Markdown Typography:** Render rich Markdown text with headings, bold, italics, monospace code blocks, unordered and numbered lists, and clickable hyperlinks.
- **Visual Pills, Badges & Custom Colors:** Inline `<pill>`, `<badge>`, `<kbd>`, and `<color #rrggbb>` formatting for cockpit annunciators, system status, and flight telemetry.
- **Intelligent Stationary Tooltips (`TimedTooltip`):** Stationary hover tooltips that do not jitter or track the mouse, feature reading-adaptive dwell timeouts, enforce layout style isolation, and fully support embedded Markdown.
- **Tactile UX Widgets:** Smooth animated `ToggleButton`, segmented `ToggleButtons`, `SliderWithReset` (with reset icon button), `PercentSlider`, mouse-wheel scroll chasers, and hyperlink widgets.
- **Universal C++ & PascalCase Alias Parity:** Strict parity with C++ Dear ImGui casing (`imgui.Markdown.Text`, `imgui.TimedTooltip.Text`, `imgui.ToggleButton`), accompanied by short aliases (`imgui.MD.Text`, `MD.Text`, `TimedTooltip.Text`) and case-tolerant fallbacks.
- **Variadic String Formatting:** Pass `printf`-style format specifiers (`%s`, `%d`, `%.2f`) directly into any text or tooltip function without needing manual calls to `string.format()`.

---

2. Markdown Extension (`imgui.Markdown` / `MD`)

The Markdown extension parses standard Markdown syntax and renders it as native Dear ImGui draw commands within the current window.

Semantic Font Auto-Wiring

The Markdown system automatically hooks into FlyWithLua's **Named Font Registry**:

Markdown Element	Target Font Name	Fallback Font	Visual Style
Heading 1 (`#`)	`Roboto-Medium-20`	`Roboto-Medium`, Default	20px Bold / Underlined
Heading 2 (`##`)	`Roboto-Medium-16`	`Roboto-Medium`, Default	16px Bold / Underlined
Heading 3 (`###`)	`Roboto-Medium-16`	`Roboto-Medium`, Default	16px Bold
Bold (`**bold**`)	`Roboto-Medium`	`Roboto-Regular`, Default	Medium / Semi-Bold weight
Italics (`*italic*`)	`Roboto-Light-16`	`Roboto-Light`, Default	Light / Slanted weight
Monospace / Code (`code`)	`ProggyForever`	`ProggyClean`, Default	Monospace vector font

Markdown Functions

`imgui.Markdown.Text(format_string, ...)`

Renders formatted Markdown text. Supports variadic arguments.

```
imgui.Markdown.Text("# Pre-Flight Inspection")
imgui.Markdown.Text("Verify engine oil pressure: **%.1f PSI** (Nominal: %d-%d PSI)", oil_psi, 45, 60)
```

`imgui.Markdown.TextWrapped(format_string, ...)`

Renders formatted Markdown text with automatic line wrapping to the current window boundary.

```
imgui.Markdown.TextWrapped("This is a long flight briefing paragraph that wraps smoothly across the window...")
```

`imgui.Markdown.TextUnformatted(string)`

Renders Markdown text directly without variadic format parsing (safe for strings containing arbitrary `%` characters).

```
imgui.Markdown.TextUnformatted("Direct text containing 100% pure data.")
```

`imgui.Markdown.ButtonMD(label, [tooltip], [wrap_width])`

Renders a button whose label is parsed with full Markdown support. Returns `true` when clicked.

```
if imgui.Markdown.ButtonMD("Confirm **Auto-Land**##btn", "Engage dual-channel CAT III approach", 300) then
    engage_autoland()
end
```

`width, height = imgui.Markdown.CalcTextSize(text, [wrap_width])`

Calculates the pixel dimensions required to render a given Markdown string before drawing.

```
local w, h = imgui.Markdown.CalcTextSize("### Heading Example")
```

---

3. Inline Styling: Pills, Badges, Keys & Colors

FlyWithLua supports inline XML-style tags embedded directly inside Markdown text:

1. Status Pills (`<pill color>TEXT</pill>`)

Pills render rounded, solid-colored badges with contrasting text -- ideal for system annunciators and warning flags.

Supported color keywords: `green`, `red`, `blue`, `yellow`, `orange`, `purple`, `gray`, `error`, `warning`, `caution`, `info`, `success`.

```
imgui.MD.Text("Hydraulics: <pill green>SYSTEM NORMAL</pill> | Fuel Tank A: <pill warning>LOW LEVEL</pill>")
```

2. Version & Topic Badges (`<badge color>TEXT</badge>`)

Badges render slightly more subtle categorized chips for metadata, build tags, and feature markers.

```
imgui.MD.Text("FlyWithLua <badge blue>v2.8.18</badge> running on <badge purple>Metal / Vulkan</badge>")
```

3. Keyboard Shortcut Tags (`<kbd>Key</kbd>`)

Renders a bordered, keycap-styled box for visual keyboard shortcuts.

```
imgui.MD.Text("Press <kbd>Shift</kbd> + <kbd>Ctrl</kbd> + <kbd>R</kbd> to reload Lua scripts.")
```

4. Custom Hex Colors (`<color #rrggbb>Text</color>`)

Colors arbitrary text spans using standard 6-digit hex color codes.

```
imgui.MD.Text("Target speed: <color #00bc8c>250 knots</color> below <color #f39c12>10,000 ft</color>.")
```

5. Clickable Hyperlinks

Standard Markdown link syntax `[Label](URL)` is supported. When clicked, FlyWithLua automatically invokes the native operating system browser (via `open` on macOS, `ShellExecute` on Windows, or `xdg-open` on Linux).

```
imgui.MD.Text("For aircraft manuals, see the [FlyWithLua Docs](https://github.com/sparker256/FlyWithLua).")
```

---

4. TimedTooltip Extension (`imgui.TimedTooltip` / `TimedTooltip`)

The standard Dear ImGui tooltip (`imgui.SetTooltip`) suffers from several UX limitations in desktop flight simulators:

- **Cursor tracking / Jitter:** The tooltip position moves with every sub-pixel mouse movement, distracting the pilot.
- **Zero Reading Intelligence:** Disappears instantly when the mouse nudges off the target, regardless of how much text remains to read.
- **Style Contamination:** Inherits whatever window padding, spacing, or color overrides happen to be active on the widget stack.

`TimedTooltip` completely resolves these issues through a dedicated tooltip controller:

1. **Stationary Anchor:** Anchors at the position where hover was initiated and remains rock-solid until dismissed.
2. **Reading-Adaptive Timeout:** Dwell duration scales dynamically based on the word count of the tooltip content.
3. **Style Isolation:** Rendered in an independent style context with clean standard padding and borders.
4. **Rich Markdown Formatting:** Supports full headings, bold, italics, pills, badges, and colors via `TimedTooltip.MD.Text`.

TimedTooltip Functions

`imgui.TimedTooltip.Text(format_string, ...)`

Shows a stationary plain text tooltip when the preceding widget is hovered.

```
imgui.Button("AP Master##btn")
imgui.TimedTooltip.Text("Flight Director and Autopilot Master Engagement Switch")
```

`imgui.TimedTooltip.MD.Text(format_string, ...)`

Shows a stationary tooltip rendered with full Markdown and pill styling.

```
imgui.Button("Pitot Heat##btn")
imgui.TimedTooltip.MD.Text("### Pitot Tube Heating System\n\n" ..
                           "Status: <pill green>ON</pill>\n\n" ..
                           "• Prevents ice accumulation in pitot static probes.\n" ..
                           "• <color #f39c12>Caution: Ground operation limited to 5 minutes.</color>")
```

Dwell & Dimension Controls

Configure tooltip behavior globally or per session:

- `imgui.TimedTooltip.SetHoverDelay(cycles)` -- Number of frames cursor must hover before tooltip appears.
- `imgui.TimedTooltip.SetMaxDisplay(cycles)` -- Base display timeout in frames.
- `imgui.TimedTooltip.SetMaxWidth(pixels)` -- Maximum wrap width in pixels (defaults to 400px).
- `imgui.TimedTooltip.SetMinWidth(pixels)` -- Minimum width in pixels.
- `imgui.TimedTooltip.Clear()` -- Immediately dismisses any active tooltip.

---

5. Modern Contrib Widgets

The `contrib/widgets/` family adds tactile, specialized controls for simulation settings and telemetry adjustment.

`changed, new_val = imgui.ToggleButton(label, bool_val)`

An animated iOS-style sliding pill toggle switch with smooth tactile color transitions.

```
local changed, new_state = imgui.ToggleButton("Cabin Lights", cabin_lights_on)
if changed then
    cabin_lights_on = new_state
end
```

`changed, new_state = imgui.ToggleButtons(label, int_state, off_label, on_label)`

A segmented multi-state toggle control.

```
local changed, new_mode = imgui.ToggleButtons("##RecogLights", light_mode, "OFF", "STROBE")
if changed then
    light_mode = new_mode
end
```

`changed, new_val = imgui.SliderWithReset(label, val, min, max, default_val)`

A horizontal slider paired with an adjacent reset button (■) that instantly returns the slider to its calibrated default value.

```
local changed, new_trim = imgui.SliderWithReset("Aileron Trim", aileron_trim, -1.0, 1.0, 0.0)
if changed then
    aileron_trim = new_trim
end
```

`changed, new_val = imgui.PercentSlider(label, val)`

A slider designed for normalized floating point values `[0.0 .. 1.0]`, automatically displayed and formatted as an integer percentage (`0% .. 100%`).

```
local changed, new_flaps = imgui.PercentSlider("Flaps Deflection", flaps_deploy)
if changed then
    flaps_deploy = new_flaps
end
```

`changed, new_val = imgui.PercentSliderWithReset(label, val, default_val)`

Combines the normalized percent slider with an automatic reset button.

```
local changed, new_mix = imgui.PercentSliderWithReset("Mixture", fuel_mixture, 1.0)
if changed then
    fuel_mixture = new_mix
end
```

`changed, new_val = imgui.AdjustItemOnMouseWheel(val, min, max, [step])`

Detects mouse wheel scrolling when hovering over the preceding item (e.g. a button or custom area) and increments or decrements the floating-point value.

```
imgui.Button(string.format("NAV 1: %.2f MHz", nav1_freq), 220, 30)
local changed, new_freq = imgui.AdjustItemOnMouseWheel(nav1_freq, 108.00, 117.95, 0.05)
if changed then
    nav1_freq = new_freq
end
```

`changed, new_idx = imgui.CycleItemOnMouseWheel(idx, count, [wrap])`

Cycles an integer index when the mouse wheel is scrolled over the preceding widget. Supports standard 1-based Lua indexing.

```
local gear_modes = { "UP", "OFF", "DOWN" }
imgui.Button(string.format("Gear Lever: %s", gear_modes[gear_idx]), 200, 30)
local changed, new_idx = imgui.CycleItemOnMouseWheel(gear_idx, #gear_modes, true)
if changed then
    gear_idx = new_idx
end
```

`clicked = imgui.ButtonLink(label, [url])` **and** `imgui.TextLink(label, [url])`

Renders a button or text item styled as a hyperlink. If `url` is supplied, clicking automatically opens the URL in the system browser.

```
imgui.TextLink("Check METAR Weather Reports", "https://aviationweather.gov/metar")
imgui.ButtonLink("FlyWithLua Forum Support", "https://forums.x-plane.org")
```

6. Universal C++ & Lua Alias Matrix

FlyWithLua provides comprehensive alias coverage matching both standard Dear ImGui PascalCase conventions and C++ namespace aliases:

Category	Primary API	Short Alias	Global Scope	Suffix Style
Markdown Text	`imgui.Markdown.Text`	`imgui.MD.Text`	`Markdown.Text` / `MD.Text`	`imgui.TextMD`
Markdown Wrapped	`imgui.Markdown.TextWrapped`	`imgui.MD.TextWrapped`	`Markdown.TextWrapped`	`imgui.TextWrappedMD`
Markdown Direct	`imgui.Markdown.TextUnformatted`	`imgui.MD.TextUnformatted`	`Markdown.TextUnformatted`	--
Markdown Button	`imgui.Markdown.ButtonMD`	`imgui.MD.ButtonMD`	`Markdown.ButtonMD`	`imgui.ButtonMD`
TimedTooltip Plain	`imgui.TimedTooltip.Text`	--	`TimedTooltip.Text`	--
TimedTooltip MD	`imgui.TimedTooltip.MD.Text`	`imgui.TimedTooltip.TextMD`	`TimedTooltip.MD.Text`	`TimedTooltip.TextMD`
Toggle Button	`imgui.ToggleButton`	`imgui.toggle_button`	--	--
Slider With Reset	`imgui.SliderWithReset`	`imgui.slider_with_reset`	--	--
Percent Slider	`imgui.PercentSlider`	`imgui.percent_slider`	--	--
Mouse Wheel Adjust	`imgui.AdjustItemOnMouseWheel`	`imgui.adjust_item_on_mouse_wheel`	--	--
Mouse Wheel Cycle	`imgui.CycleItemOnMouseWheel`	`imgui.cycle_item_on_mouse_wheel`	--	--

---

7. Complete Interactive Script Recipe

Here is a minimal, complete script showing all extensions working together:

```
if not SUPPORTS_FLOATING_WINDOWS then return end

local wnd = float_wnd_create(540, 420, 1, true)
float_wnd_set_title(wnd, "Avionics Contrib Demo")
float_wnd_set_imgui_builder(wnd, "build_avionics_wnd")

local ap_master = true
local altitude_target = 10000

function build_avionics_wnd(wnd, x, y)
    -- Markdown Header with Pill
    MD.Text("# Flight Management System <badge blue>ACTIVE</badge>")
    MD.Text("Current status: <pill green>EN ROUTE</pill> | Baro: **29.92 inHg**")
    imgui.Separator()

    -- Animated Toggle with TimedTooltip
    local changed, new_ap = imgui.ToggleButton("Autopilot Master", ap_master)
    if changed then ap_master = new_ap end
    TimedTooltip.MD.Text("### Autopilot Master Control\n" ..
                         "Toggles coupled lateral and vertical flight directors.\n" ..
                         "<color #00bc8c>Safe to disengage during cruise flight.</color>")

    -- Altitude Target with Mouse Wheel Tuning
    imgui.Spacing()
    imgui.Button(string.format("Target Altitude: %5d ft (Scroll Wheel)##alt", altitude_target), 300, 30)
    local alt_changed, new_alt = imgui.AdjustItemOnMouseWheel(altitude_target, 0, 45000, 500)
    if alt_changed then altitude_target = math.floor(new_alt) end
    TimedTooltip.Text("Scroll mouse wheel over button to adjust target altitude in 500 ft steps.")

    -- Hyperlink to Documentation
    imgui.Spacing()
    imgui.Separator()
    imgui.TextLink("FlyWithLua Online Documentation", "https://github.com/sparker256/FlyWithLua")
end
```